

OBLIGATORIO

BASE DE DATOS 2

SANTIAGO TRINIDAD

DIEGO GAMARRA

RODRIGO PEREIRA

22/06/2026

ÍNDICE

Contenido

INTRODUCCIÓN	2
OBJETIVOS	2
MARCO TEÓRICO	3
DECISIONES DE DISEÑO DE ACUERDO CON LA PROPUESTA	5
ALTERNATIVAS DE SISTEMAS YA EXISTENTES	6
TECNOLOGÍAS UTILIZADAS.....	7
MODELO DE DATOS – EVOLUCIÓN DIAGRAMA ENTIDAD RELACIÓN	9
MODELO LÓGICO	13
SCRIPTS DE CREACIÓN DE LA BASE DE DATOS (DDL)	16
IMPLEMENTACIÓN	17
CONCLUSIONES	19
REFERENCIAS BIBLIOGRÁFICAS.....	20

INTRODUCCIÓN

En este informe, se documenta el análisis, diseño e implementación de un “prototipo” de sistema integral de ticketing para eventos de alta concurrencia, en el contexto de los partidos del Mundial 2026 a disputarse en Estados Unidos, Canadá y México.

A diferencia de un sistema de ticketing tradicional basado en entradas estáticas, la plataforma propuesta implementa un modelo de entrada dinámica, el archivo digital no es una imagen estática, sino un token que se regenera periódicamente para prevenir el fraude y la reventa no autorizada, manteniendo en todo momento un registro histórico de esta.

El objetivo del trabajo es relevar los requerimientos, evaluar soluciones existentes en el mercado, modelar el dominio del problema, construir la base de datos relacional en SQL en modalidad “Cliente/Servidor”, y desarrollar una aplicación que cubra la funcionalidad mínima exigida, registro de usuarios, compra, transferencia y validación de entradas, gestión de eventos, entre otros.

OBJETIVOS

- Modelar el dominio mediante un Modelo Entidad Relación (MER) que pueda ser extensible en el futuro.
- Implementar dicho modelo en la base de datos (MySQL).
- Desarrollar una aplicación DEMO cliente/servidor.
- Garantizar las reglas de negocio predefinidas por la letra del obligatorio.

MARCO TEÓRICO

En esta sección, haremos un resumen de los fundamentos teóricos utilizados durante el trabajo. Se abordan conceptos vistos en clase (curso Base de Datos 1, Base de Datos 2), así como también conceptos adquiridos en la web.

Modelo Entidad Relación

El Modelo Entidad Relación, propuesto por Chen, es una herramienta de modelado conceptual que representa el dominio mediante entidades, atributos y relaciones de manera conceptual, previo a pensar en el modelo de la base de datos (tablas).

El modelo relacional, propuesto por Codd, organiza los datos en relaciones (tablas) compuestas por tuplas y atributos, donde las asociaciones se expresan mediante claves primarias y foráneas.

El pasaje del MER al modelo lógico relacional es un proceso en el que las entidades fuertes, entidades débiles, jerarquías y relaciones de nuestro dominio, dependiendo de la cardinalidad, se transforman en tablas y restricciones.

Normalización

La normalización es el proceso de organizar los atributos y relaciones para reducir la redundancia y evitar posibles problemas de inserción, actualización y borrado en nuestras bases de datos.

Las formas normales más utilizadas son la Primera (1FN), que elimina los grupos repetitivos y los atributos multivaluados, la Segunda (2FN), que elimina dependencias parciales respecto de claves compuestas, y la Tercera (3FN), que elimina dependencias transitivas.

En el modelo propuesto, por ejemplo, los teléfonos del usuario se modelan como un atributo multivaluado que, en el pasaje al modelo relacional, es separado en una

nueva tabla, para cumplir la 1FN. Por otro lado, la dirección se la modela como un atributo compuesto. Este se descompone en sus elementos atómicos.

Restricciones e Integridad en tablas

En esta oportunidad, hemos modelado e implementado en nuestro MER y en nuestro DDL, varias restricciones que se han modelado mediante claves primarias (PK), claves foráneas (FK), restricciones (checks) y restricciones de unicidad.

Las reglas de negocio que no puedan ser restringidas mediante los procedimientos descritos anteriormente, deben ser controladas mediante triggers o procedimientos implementados desde aplicación. Por ejemplo, el control del aforo por sector o la cantidad de veces que una entrada puede ser transferida a un usuario, deben ser controladas a través de algunos de los mecanismos anteriores.

Control de Acceso basado en Roles (RBAC)

El control basado en roles asigna permisos a roles que son luego asignados a usuarios determinados.

El sistema implementa tres roles: Administrador por País Sede, Funcionario de Validación y Usuario General.

Cada rol tiene permisos específicos y el acceso a las funcionalidades está controlado en el servidor según el rol activo en la sesión.

DECISIONES DE DISEÑO DE ACUERDO CON LA PROPUESTA

- Para evitar que dos usuarios compren el mismo asiento, se utiliza la instrucción SELECT ... FOR UPDATE, que “bloquea” las filas seleccionadas hasta que la transacción finaliza. Esto garantiza que la verificación de disponibilidad y la creación de la entrada sean una operación atómica, es decir, nos aseguramos que hasta que el usuario A no termine su compra por ejemplo, nadie más puede tocar dicho asiento.
- Para dificultar la falsificación o reutilización de entradas, el código QR no es estático, sino dinámico. Se genera un token único cada 30 segundos. Al escanear, el sistema verifica que el token esté activo y no haya vencido. Esto impide que alguien capture el QR de otra persona y lo use más tarde.
- Las entradas pueden transferirse entre usuarios, pero con un límite de 3 transferencias por entrada. Cada transferencia queda registrada con origen, destino y estado, formando un historial (log) completo de qué usuario tuvo la entrada asignada.
- La tasa de comisión sobre las ventas no es fija. Se almacena en una tabla “Tasa_Comision” con rangos de fechas, de modo que al calcular el monto de una venta se aplica la tasa de ese momento. Esto permite modificar la comisión en el futuro sin afectar las ventas históricas.

ALTERNATIVAS DE SISTEMAS YA EXISTENTES

1. RedTickets

RedTickets es una plataforma de ticketing uruguaya que cubre una amplia variedad de eventos, como lo pueden ser teatro, música, deportes, fútbol, festivales, conferencias, etc.

Cuenta con una aplicación móvil disponible en App Store y Google Play, lo que permite comprar y gestionar entradas desde el celular.

Para nuestro caso, RedTickets cubre los conceptos “básicos” de venta de entradas y gestión de eventos, pero no está diseñada para la escala ni la complejidad operativa de un torneo como el Mundial, de acuerdo con nuestros conocimientos y pruebas como usuarios.

Teniendo en cuenta los requerimientos del sistema descrito en la letra del obligatorio, observamos que, RedTickets no tiene un modelo de roles diferenciado por jurisdicción geográfica, no implementa QR dinámico con vencimiento por token, y no brinda información sobre el control de concurrencia en la compra o trazabilidad de transferencias de entradas entre usuarios.

2. Tickantel

Tickantel es la plataforma de venta de entradas operada por Antel. Permite comprar entradas de forma web o aplicación móvil, o de forma presencial en puntos de venta físicos. Al igual que RedTickets, cubre eventos de teatro, música, deportes, danza, carnaval y más, y acepta múltiples medios de pago. Opera como intermediario de los organizadores de eventos, es decir, comercializa y facilita la venta de entradas.

Para nuestro caso de uso, Tickantel cubre el flujo básico de compra, pero no implementa funcionalidades avanzadas relevantes al obligatorio; no tiene transferencia de entradas entre usuarios, no tiene QR dinámico, y no tiene un sistema de roles diferenciados (administrador por país, funcionario de validación).

3. Ticketmaster (USA)

Ticketmaster es la plataforma de venta de entradas más grande del mundo con foco en Estados Unidos. Es usada en conciertos, partidos de fútbol, eventos deportivos y espectáculos.

Permite comprar entradas online, elegir el asiento en un mapa, transferir entradas a otros usuarios y presentar un código QR al ingresar. También, de acuerdo con lo recabado en la web, cuenta con un sistema “anti-reventa” que detecta bots e restricciones dentro de la cuenta para garantizar esto.

La principal ventaja es su rendimiento. Opera con millones de transacciones de forma simultánea. La desventaja es que es un sistema cerrado y muy costoso de licenciar.

TECNOLOGÍAS UTILIZADAS

Para implementar el sistema decidimos usar MySQL como motor de base de datos y Python con Flask como framework web.

MySQL es un Sistema de Gestión de Bases de Datos Relacional que permite almacenar, organizar y administrar grandes volúmenes de información de manera eficiente. Utiliza el lenguaje SQL (Structured Query Language) como mecanismo principal para la definición, consulta y manipulación de datos.

Este motor de base de datos se caracteriza por su alto rendimiento, facilidad de uso, escalabilidad y amplia adopción tanto en entornos académicos como empresariales.

MySQL implementa el modelo relacional, en el cual los datos se organizan en tablas compuestas por filas y columnas, permitiendo establecer relaciones entre ellas mediante claves primarias y foráneas.

JUSTIFICACIÓN

Luego de varias discusiones y pruebas, en el grupo hemos decidido utilizar Python con la integración de Flask, ya que permite crear APIs REST de forma rápida.

En un inicio, habíamos comenzado con C# y .NET, integrándolo con el framework Entity Framework (EF), que luego de varias pruebas y trabajos, como grupo terminamos descartando ya que lo que teníamos implementado con EF realizaba las consultas SQL de forma propia, sin que nosotros tuviéramos que escribirlo, y se nos comentó en clase que no era la idea del Obligatorio y no se podía usar.

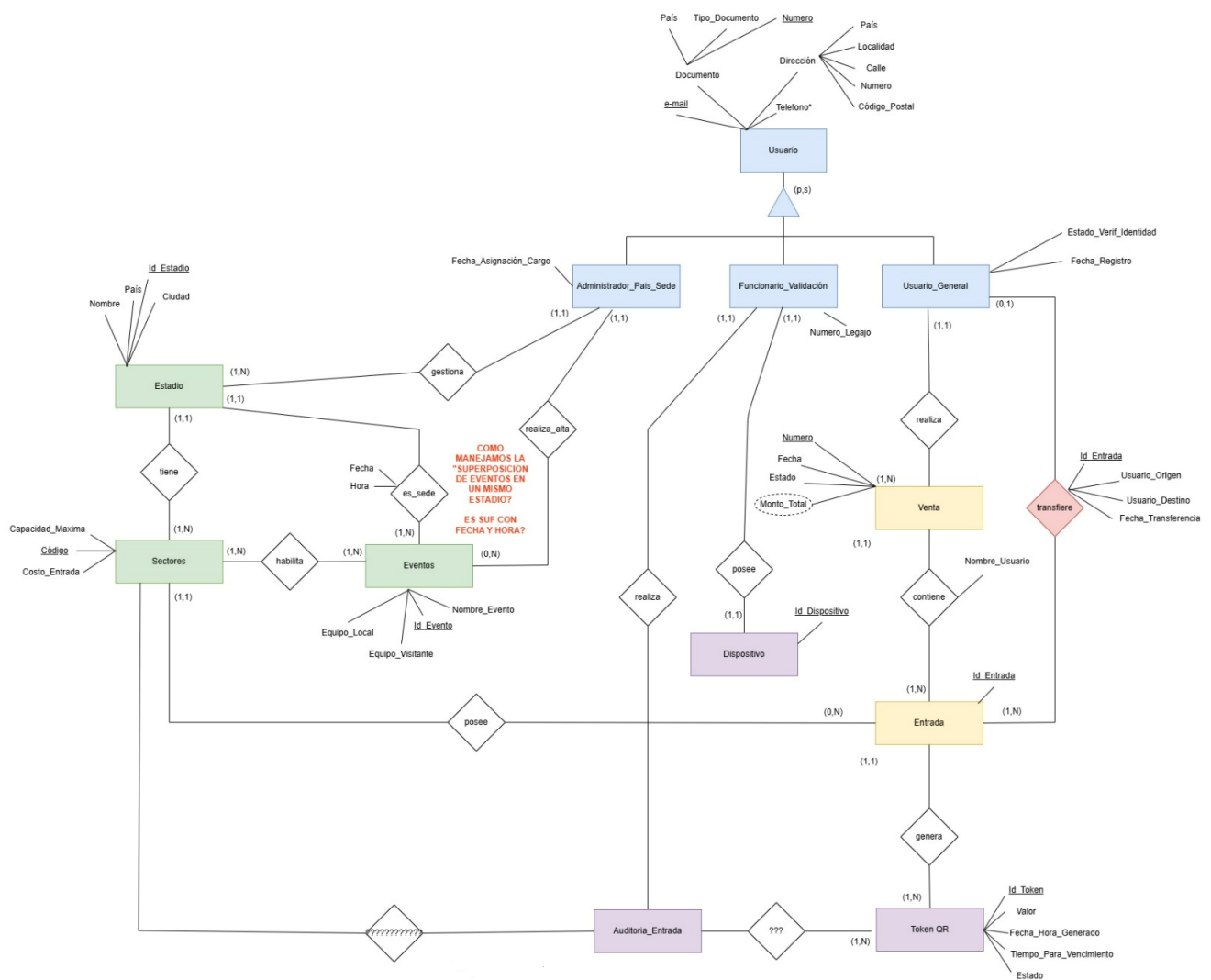
Por esta razón, la combinación MySQL + Flask nos permite conectar la base de datos directamente utilizando la librería de Python, *mysql-connector-python*. Desde allí pudimos ejecutar consultas SQL explícitas (sin ORM), y tener control total sobre las transacciones, lo que es importante para el control de concurrencia en la venta de entradas, por ejemplo.

Se optó por HTML y JavaScript puro, sin frameworks adicionales, priorizando la simplicidad, ya que no nos quedaba mucho tiempo.

MODELO DE DATOS – EVOLUCIÓN DIAGRAMA ENTIDAD RELACIÓN

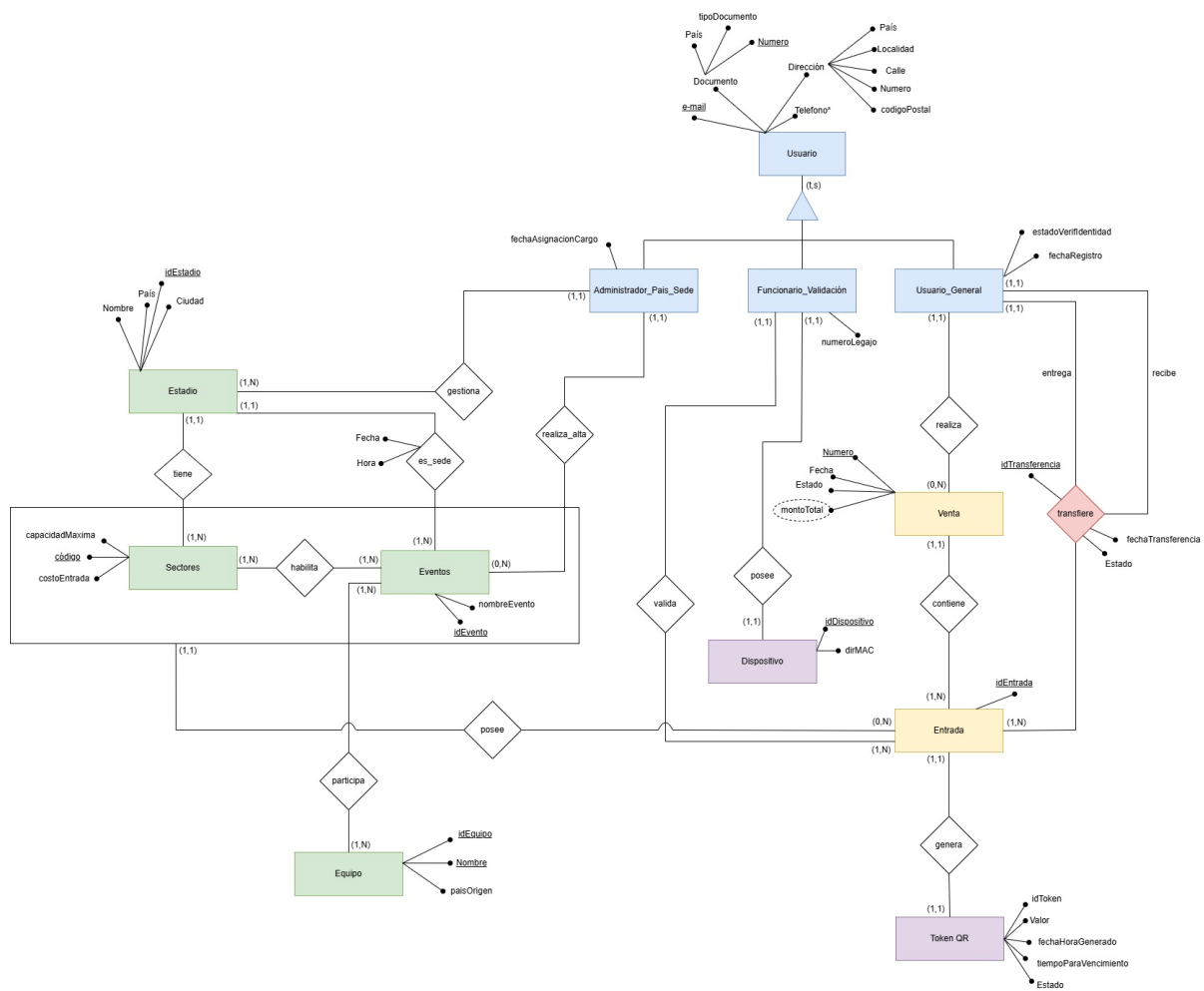
El diagrama Entidad Relación (MER), fue evolucionando a lo largo del proyecto a medida que fuimos entendiendo mejor los requerimientos y trabajando en la implementación del pasaje a tabla y armado de la app. Principalmente, nuestro modelo de datos, pasó por tres versiones antes de llegar a la versión final.

MER Versión 1



En esta primera versión de nuestro MER, diseñamos de acuerdo con la descripción de la letra del obligatorio, el esqueleto inicial del diagrama, en donde debimos tomar varias decisiones conceptuales y supuestos para intentar representar la realidad de la propuesta de la forma más coherente posible.

MER VERSION 2 (Entrega Intermedia - MER OBL)

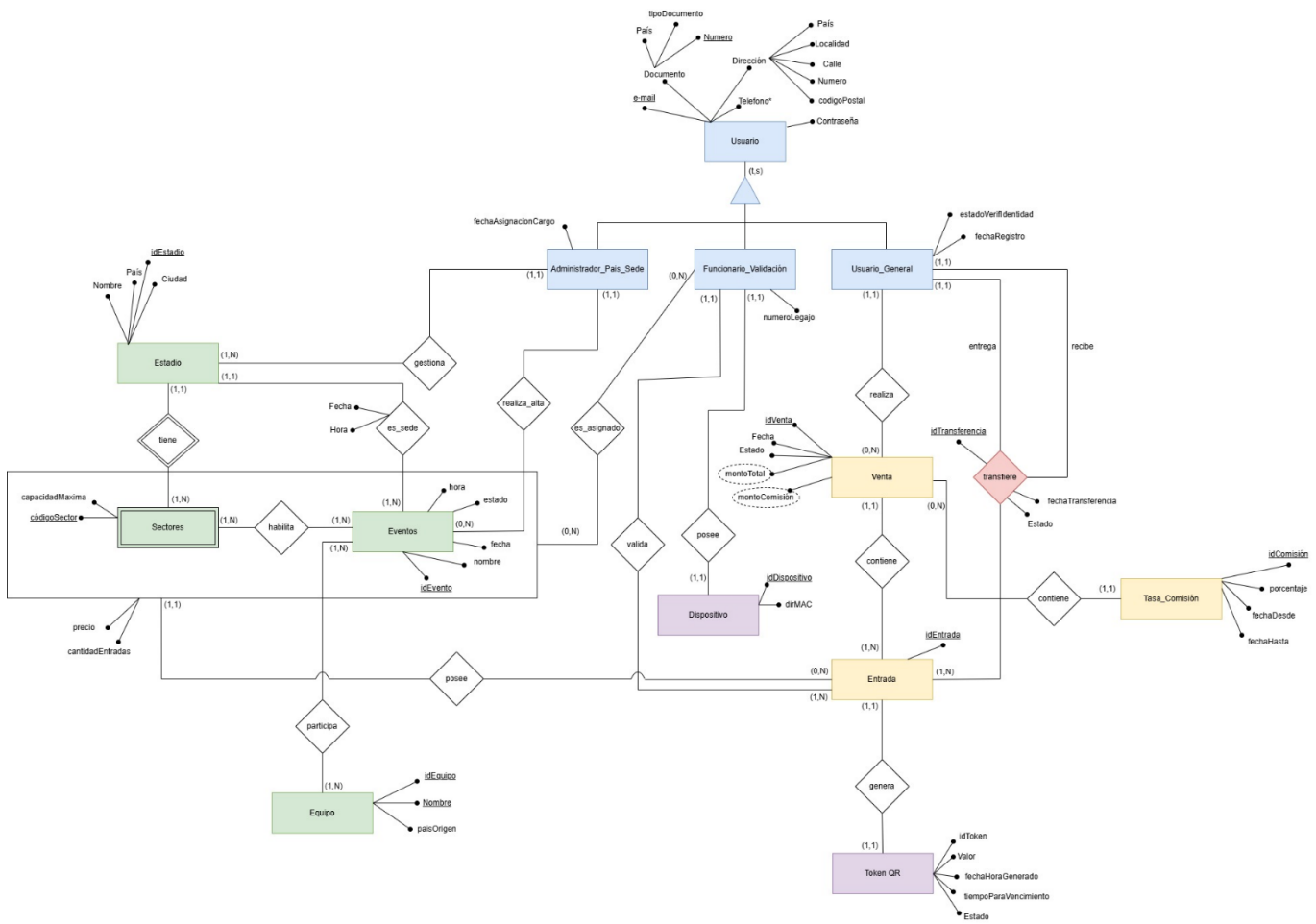


En esta versión, que fue la versión del MER que entregamos en la entrega intermedia, corregimos varios aspectos:

- Cobertura de generalización en la entidad Usuario. Ahora es total, lo que garantiza que todo usuario registrado pertenezca a al menos uno de los tres subtipos. Esto refleja correctamente la regla de negocio, no puede existir un usuario sin rol en el sistema. A la vez, es superpuesta, ya que puede existir un Administrador o un Funcionario, que a la vez, este sea Usuario General.
- Equipo, pasa a ser una entidad. En la versión 1 de nuestro MER, se habían modelado como dos atributos de la entidad Evento. En esta versión, se modeló como una entidad aparte, ya que se desea conocer la información de cada equipo de forma independiente al evento.

- La relación Transferencia, pasa a representarse como una relación ternaria.
- Se le agrega a la entidad Dispositivo el atributo dirMac para identificar de forma exacta y el dispositivo de cada funcionario.
- Se incorpora una relación llamada “Valida” entre Funcionario_Validación y Entrada.
- Se agrega también una agregación sobre la relación habilita entre Sectores y Eventos. Esto permite que la entidad Entrada se vincule no a un sector suelto ni a un evento suelto, sino a la combinación específica de un sector habilitado dentro de un evento. En el modelo relacional final esto se traduce en la tabla Evento_Sector, que actúa como clave foránea en Entrada y garantiza la integridad de esta, no se puede registrar una entrada en un sector que no esté habilitado para ese evento.
- Se decide quitar la entidad “Auditoria_Entrada” ya que estaba relacionado con el token QR y no con la entrada, y no hacía mucho sentido para el contexto. En esta versión, se crea la relación “valida” entre el funcionario y la entidad Entrada.
- Había también inconsistencias en el modelo ya que como lo hemos mencionado en el punto anterior, existía la entidad “Auditoria_Entrada”, que esta se relacionaba con el sector, lo cual no tenía mucho sentido ya que el sector, se relaciona con la entrada en si debido a que la entrada contiene un sector para poder existir.

MER VERSION 3



- En esta versión, luego de ya trabajar bastante en el proyecto, habiendo realizado el pasaje a tablas de nuestro modelo, y haber comenzado a desarrollar nuestra app, hemos concluido en esta versión final.
- Aquí, mantuvimos la relación ternaria “transfiere” entre la entidad Usuario General y Entrada, y decidimos representarla en el modelo lógico como tabla.
- Representamos la entidad Sectores débil respecto a Estadio, ya que si no existe el Estadio, no existe el Sector.
- Agregamos la entidad Tasa_Comision, ya que por letra, el precio de venta de cada entrada se calcula por el precio unitario de cada una más una tasa fija de comisión que es predefinida en un periodo de tiempo y puede cambiar.

- A la vez, se agrega una relación entre la entidad Funcionario_Validación y la agregación que contiene a las entidades Sectores y Eventos. Aquí se quiere representar que un funcionario es asignado a un Sector en un evento particular, cumpliendo el requisito de que “un funcionario debe haber validado entradas en todos los sectores a los que fue asignado durante un evento”.
- Para realizar el inicio de sesión de nuestra app, debimos agregar un atributo password a la entidad Usuario para poder realizarlo. La misma utiliza una encriptación HASH con el algoritmo SHA2, visto en una de las presentaciones de seguridad en clase. Esto garantiza que si alguien utiliza algún cliente de base de datos, y logra ingresar a consultar la tabla Usuario, no logre descifrar la contraseña de cada usuario.

MODELO LÓGICO

A continuación, se describe el modelo lógico (pasaje a tablas) resultante de transformar el MER al modelo relacional:

Usuario (email, docPais, docTipo, docNumero, dirPais, dirLocalidad, dirCalle, dirNumero, dirCodigoPostal, passw)

→ PK: email

Usuario_Telefono (email, telefono)

→ PK: email, telefono

→ FK: email → Usuario.email

Administrador_Pais_Sede (email, paisJurisdiccion, fechaAsignacionCargo)

→ PK: email

→ FK: email → Usuario.email

Funcionario_Validacion (email, numeroLegajo)

→ PK: email

→ FK: email → Usuario.email

Usuario_General (email, fechaRegistro, estadoVerifIdentidad)

→ PK: email

→ FK: email → Usuario.email

Estadio (idEstadio, nombre, pais, ciudad, emailAdmin, fechaAsignacion)

→ PK: idEstadio

→ FK: emailAdmin → Administrador_Pais_Sede.email

Sector (idSector, idEstadio, codigo, capacidadMaxima, costoEntrada)

→ PK: idSector

→ FK: idEstadio → Estadio.idEstadio

Equipo (idEquipo, nombre, paisOrigen)

→ PK: idEquipo

Evento (idEvento, nombreEvento, fecha, hora, idEstadio, emailAdmin)

→ PK: idEvento

→ FK1: idEstadio → Estadio.idEstadio

→ FK2: emailAdmin → Administrador_Pais_Sede.email

Evento_Equipo (idEvento, idEquipo, rol)

→ PK: idEvento, idEquipo

→ FK1: idEvento → Evento.idEvento

→ FK2: idEquipo → Equipo.idEquipo

Evento_Sector (idEvento, idSector)

- PK: idEvento, idSector
- FK1: idEvento → Evento.idEvento
- FK2: idSector → Sector.idSector

Venta (idVenta, numero, fechaVenta, estado, montoTotal, emailComprador)

- PK: idVenta
- FK: emailComprador → Usuario_General.email

Entrada (idEntrada, idVenta, idEvento, idSector, emailPropietario, estado, cantTransferencias)

- PK: idEntrada
- FK1: idVenta → Venta.idVenta
- FK2: (idEvento, idSector) → Evento_Sector.idEvento, Evento_Sector.idSector
- FK3: emailPropietario → Usuario_General.email

Transferencia (idTransferencia, idEntrada, emailOrigen, emailDestino, fechaTransferencia, estado)

- PK: idTransferencia
- FK1: idEntrada → Entrada.idEntrada
- FK2: emailOrigen → Usuario_General.email
- FK3: emailDestino → Usuario_General.email

Token_QR (idToken, idEntrada, valor, fechaHoraGenerado, tiempoVencimiento, estado)

- PK: idToken
- FK: idEntrada → Entrada.idEntrada

Dispositivo (idDispositivo, dirMAC, alias, emailFuncionario)

- PK: idDispositivo
- FK: emailFuncionario → Funcionario_Validacion.email

Validacion_Entrada (idValidacion, idEntrada, idToken, idDispositivo, emailFuncionario, fechaHoraValidacion)

- PK: idValidacion
- FK1: idEntrada → Entrada.idEntrada
- FK2: idToken → Token_QR.idToken
- FK3: idDispositivo → Dispositivo.id_Dispositivo
- FK4: emailFuncionario → Funcionario_Validacion.email

Asignacion_Funcionario (idEvento, idSector, emailFuncionario)

- PK: idEvento, idSector, emailFuncionario
- FK1: idEvento → Evento.idEvento
- FK2: idSector → Sector.idSector
- FK3: emailFuncionario → Funcionario_Validacion.email

Tasa_Comision (idTasa, porcentaje, fechaDesde, fechaHasta)

- PK: idTasa

SCRIPTS DE CREACIÓN DE LA BASE DE DATOS (DDL)

Se adjunta archivo junto con este informe dentro de la carpeta el archivo “**DDL OBL2.sql**”, que contiene el script de creación de tablas en MySQL, y los scripts INSERT, con algunos datos que hemos usado de prueba para testear nuestra aplicación.

Nota: En el archivo “DDL OBL2.SQL”, además de los scripts de creación de tablas con sus PK y FK correspondientes, se agregaron también instancias de CHECK, en donde se controlan varias reglas de negocio a través de Base de Datos.

Por ejemplo, se chequea que una venta no tenga asociada más de 5 entradas, los sectores de los estadios sean A, B, C, D, como se describe en la letra, etc.

IMPLEMENTACIÓN

Funcionalidades implementadas

Gestión de usuarios y roles

Los usuarios se registran con sus datos personales y pueden tener uno o más roles. El control de acceso es por sesión, es decir, depende del rol que tenga el usuario que desea ingresar. Al iniciar sesión, se almacena el email y los roles activos, y cada pantalla y endpoint verifica que el usuario tenga el rol necesario antes de ejecutar cada operación. Esto quiere decir que, al ser un sistema parametrizado en roles, no todos los usuarios van a poder visualizar enteramente todo el sistema, sino que van a poder ver solamente lo que el rol le permita.

Gestión de infraestructura y eventos

El administrador puede crear estadios y sectores dentro de su jurisdicción geográfica, crear eventos en sus estadios, vincular equipos y habilitar sectores para la venta. El sistema impide la superposición de eventos en un mismo estadio con menos de 4 horas de diferencia. Cada administrador solo puede ver y gestionar los estadios y eventos de su propio país.

Venta de entradas

El usuario está habilitado a comprar entre una y cinco entradas por transacción. El sistema verifica la disponibilidad del sector con un lock a nivel de base de datos con el mecanismo de SELECT ... FOR UPDATE. Con esto, logramos evitar el posible sobre aforo en condiciones de alta concurrencia.

El monto total se calcula aplicando la tasa de comisión vigente en la tabla Tasa_Comision. La venta inicialmente se inicializa con el estado “pendiente”, pasa a estado “confirmada” cuando el usuario declara haber pagado, y al estado “paga” cuando el administrador confirma la recepción del pago.

Transferencias

Un usuario puede transferir una entrada a otro usuario registrado como máximo tres veces. La transferencia queda en estado “pendiente” hasta que el destinatario la acepta o rechaza. Si la acepta, la entrada cambia de propietario. Como lo hemos mencionado anteriormente, el sistema impide transferir una entrada más de 3 veces y valida que el destinatario exista en el sistema antes de registrar la solicitud de transferencia.

QR Dinámico

Cuando el usuario va a presentar su entrada, la aplicación solicita al servidor un token nuevo con “tiempo de vida” de 30 segundos. El servidor genera un código de token, lo almacena en la tabla Token_QR con su vencimiento y lo devuelve.

El frontend muestra el QR y lo renueva automáticamente antes de que expire. Al escanear, el funcionario valida el token contra la base de datos, se verifica que esté activo, no haya vencido y corresponda a una entrada válida. Si todo es correcto, la entrada pasa a estado “consumida” y se registra la validación con el que el funcionario realizó la validación y también queda registro de cual fue el dispositivo que este utilizó para realizar la validación.

Cobertura de funcionarios en sector asignado

Cada funcionario está asignado a uno o más sectores de un evento. El sistema registra en la tabla Validacion_Entrada quién validó cada entrada, lo que permite generar un reporte de cobertura. Se registra cuántas entradas validó cada funcionario en cada sector asignado, cuantas les falta por validar, y si completó la cobertura total del evento.

Reportes

El administrador puede consultar el ranking de eventos con más entradas vendidas, el ranking de mayores compradores, y el reporte de cobertura de funcionarios por evento con el progreso de escaneo por sector.

CONCLUSIONES

El trabajo permitió diseñar e implementar un sistema de ticketing con requerimientos de negocio reales, predefinidas. Este cuenta con funcionalidades como el control de acceso por roles, concurrencia en la compra de entradas, trazabilidad de transferencias de entradas entre distintos usuarios, tokens QR con vencimiento dinámico y comisión variable en el tiempo.

Desde el lado de la base de datos, el desafío más interesante fue diseñar el modelo para que reflejara consistentemente las reglas de negocio sin redundancia. Algunos ejemplos pueden ser el usar la agregación sobre Evento_Sector para que cada entrada quede vinculada a un sector habilitado específicamente para ese evento y no a cualquiera que sea posible, ya que para dicho evento, podría estar deshabilitado.

La evolución del MER en sus tres versiones también fue un aprendizaje importante. Observando hacia atrás y analizando todo el proyecto, nos damos cuenta de que el modelo no nace completo, sino que se va refinando a medida que aparecen casos de uso que el modelo original no cubre correctamente.

En cuanto a la implementación, el uso de SELECT ... FOR UPDATE fue clave para garantizar que la verificación de disponibilidad y la escritura de la entrada sean atómicas. Sin ese bloqueo, dos usuarios comprando al mismo tiempo podrían superar la capacidad del sector.

[LINK REPOSITORIO PROYECTO GITHUB](#)

REFERENCIAS BIBLIOGRÁFICAS

- MySQL AB. (2024). MySQL 8.0 Reference Manual. Oracle Corporation.
<https://dev.mysql.com/doc/refman/8.0/en/introduction.html>
- RedTickets. (2024). *Plataforma de venta de entradas online*. <https://redtickets.uy/>
- Tickantel. *Plataforma de venta de entradas online*. <https://tickantel.com.uy/inicio/>
- Ticketmaster. (2024). Ticketmaster developer platform.
<https://www.ticketmaster.com/>
- Presentaciones y apuntes de Clase – Curso Base de Datos II – Prof. Ximena Romano.
- FIFA 2026. How does the Ticket Transfer feature work? -
<https://gpcustomersupportfwc2026.tickets.fifa.com/hc/en-gb/articles/30547624797341-1-How-does-the-Ticket-Transfer-feature-work>